Generating Time-Correlated Distribution

Sanjeev K

(PH25MSCST11026)

Question

1. Generating Time-Correlated Gaussian Distribution

a) Generate a set of 1000 random numbers drawn from a Gaussian distribution with zero mean and unit variance.

b) Using this initial set, generate consecutive sets of 1000 numbers each, ensuring that each new set is correlated with the previous one. The correlation should decay exponentially with a characteristic correlation time τc.

c) Compute and plot the autocorrelation function across different sets to verify the expo- nential decay.

A Gaussian (normal) distribution is defined as

$$F(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where,

$\mu \rightarrow$ mean (center of distribution)
$\sigma \rightarrow$ standard deviation (spread)

Importing required libraries

In [439…

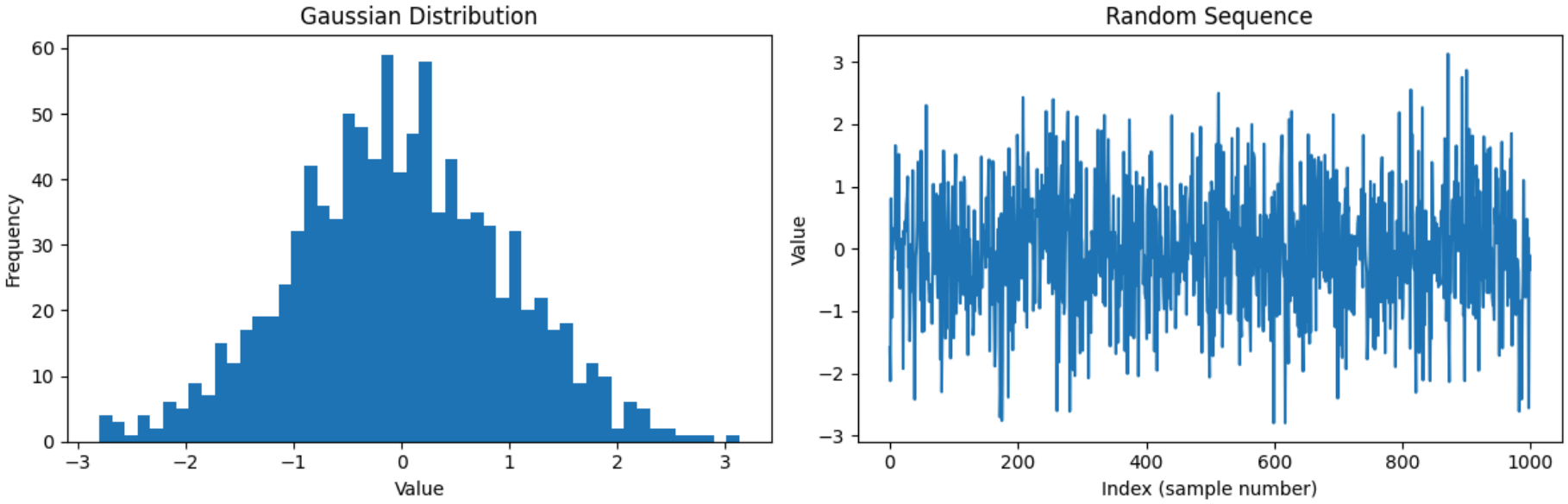
```
#imports
import numpy as np
import matplotlib.pyplot as plt
```

PART (A)

In [440…

```
#Generating a set of 1000 random numbers drawn from a Gaussian distribution with zero mean and unit variance
x = np.random.normal(0, 1, 1000)

#plot histogram
fig, axes = plt.subplots(1, 2, figsize=(12,4))
axes[0].hist(x, bins=50)
axes[0].set_title("Gaussian Distribution")
axes[0].set_xlabel("Value")
axes[0].set_ylabel("Frequency")

#Time series plot
axes[1].plot(x)
axes[1].set_title("Random Sequence")
axes[1].set_xlabel("Index (sample number)")
axes[1].set_ylabel("Value")
```

```
plt.tight_layout()
plt.show()
```



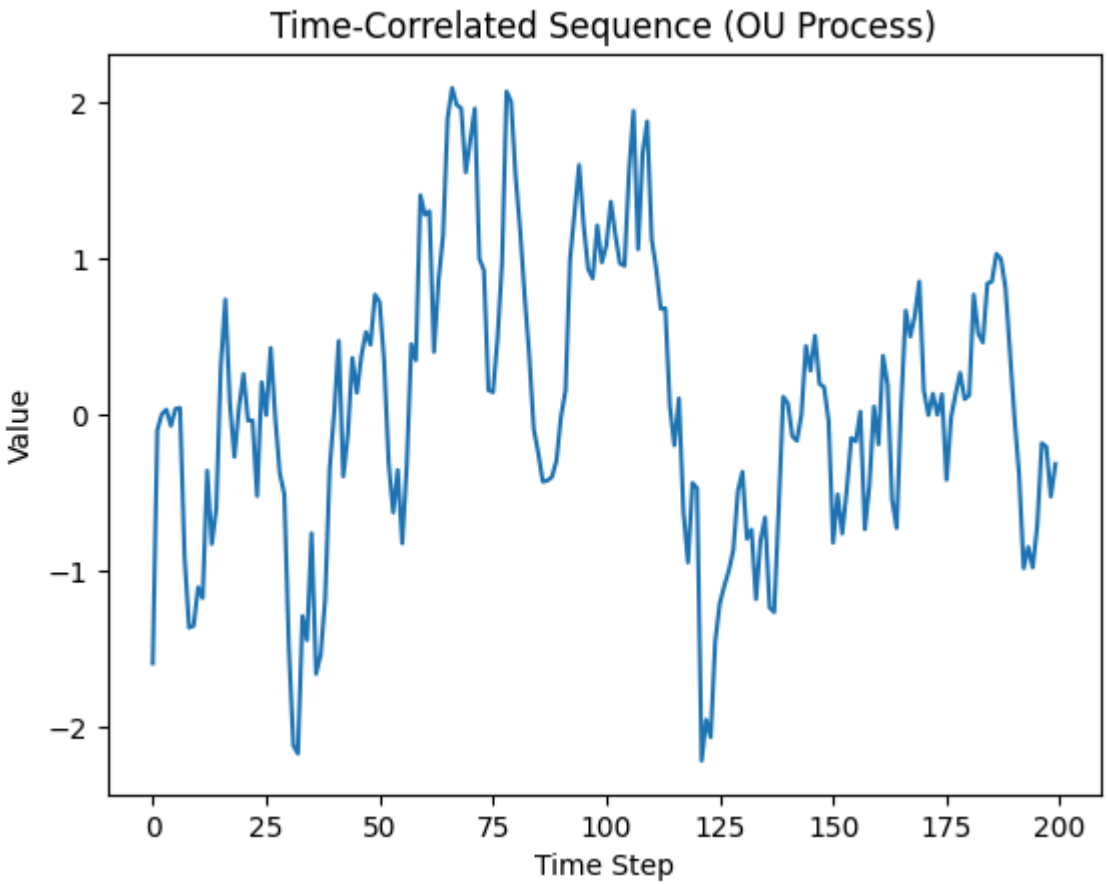
> A set of 1000 random numbers was generated from a standard normal distribution using `np.random.normal(0,1,1000)`. The generated data have mean approximately zero and unit variance. The histogram matches the theoretical Gaussian probability density function, confirming correct generation of white noise.

PART (B)

In [441…

```
initial_set  = x
def Generating_sets(initial_set, N, T, tau, dt=1):
```

```
    time_steps = int(T / dt)
    VX = np.zeros((time_steps, N))
    VX[0] = initial_set

    for t in range(1, time_steps):
        dW = np.random.normal(0, np.sqrt(dt), N)
        VX[t] = VX[t-1] - (VX[t-1] / tau) * dt + np.sqrt(2 / tau) * dW

    return VX

#parameters
N = 1000
T = 20
tau = 1
dt = 0.1
VX = Generating_sets(initial_set, N, T, tau, dt)

#plot
plt.plot(VX[:,0])
plt.title("Time-Correlated Sequence (OU Process)")
plt.xlabel("Time Step")
plt.ylabel("Value")
plt.show()
```

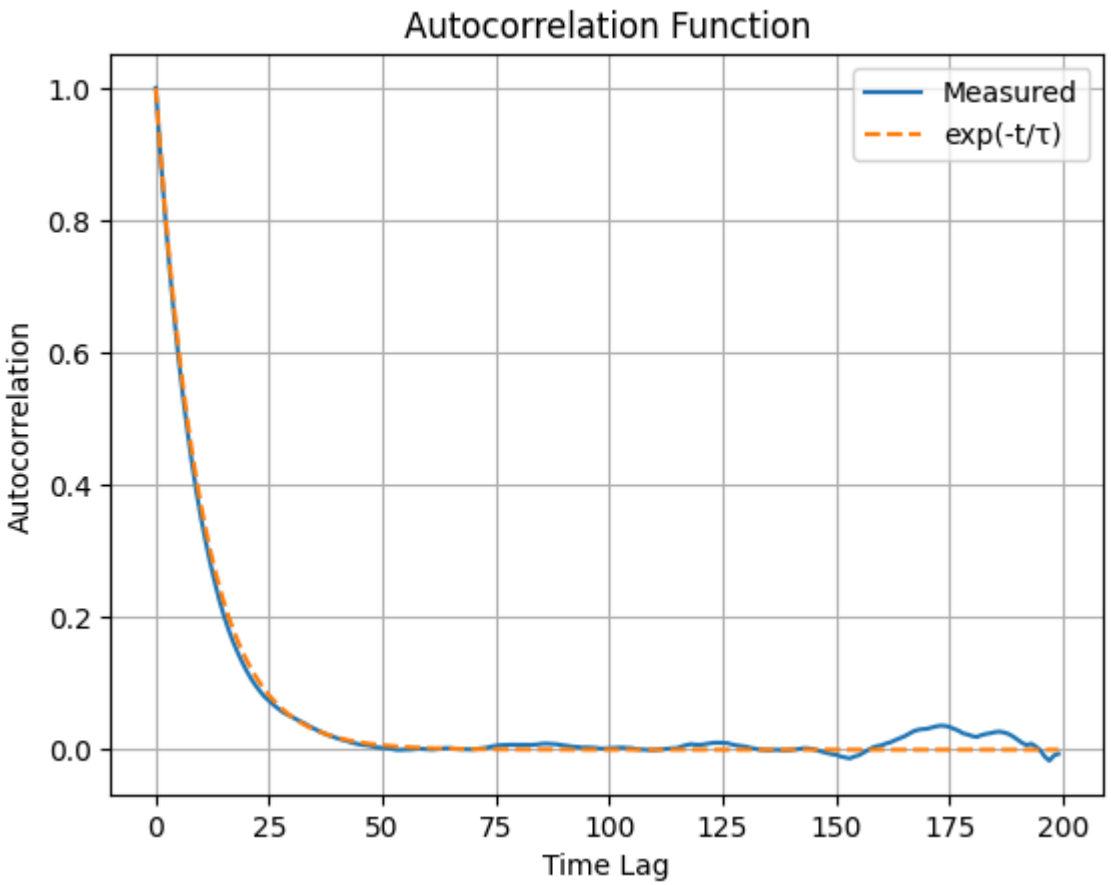


PART (C)

In [451…

```
def autocorrelation_ou(VX, max_lag_steps):
    X_mean = np.mean(VX, axis=1, keepdims=True)
    X_fluct = VX - X_mean
    denom = np.mean(X_fluct**2)

    AC = np.array([
        np.mean(X_fluct[:-t] * X_fluct[t:]) if t > 0 else denom
        for t in range(max_lag_steps)
```

```
    ])

    return AC / denom

#parameters
max_lag_time = 200
max_lag_steps = int(max_lag_time / dt)
time_lags = np.arange(max_lag_steps) * dt

AC_vec = autocorrelation_ou(VX, max_lag_steps)
theory_AC = np.exp(-time_lags / tau)

#plot autocorrelation
plt.plot(time_lags, AC_vec, label="Measured")
plt.plot(time_lags, theory_AC, '--', label="exp(-t/τ)")
plt.xlabel("Time Lag")
plt.ylabel("Autocorrelation")
plt.title("Autocorrelation Function")
plt.legend()
plt.grid()
plt.show()
```



Question

2. Generating Time-Correlated Bimodal Distribution

2. Generating Time-Correlated Bimodal Distribution (a) Generate a set of 1000 random numbers following a bimodal distribution, sampled from a mixture of two Gaussian distributions:

$$P(x) = w_1 N(\mu 1, \sigma_1^2) + w_2 N(\mu 2, \sigma_2^2)$$

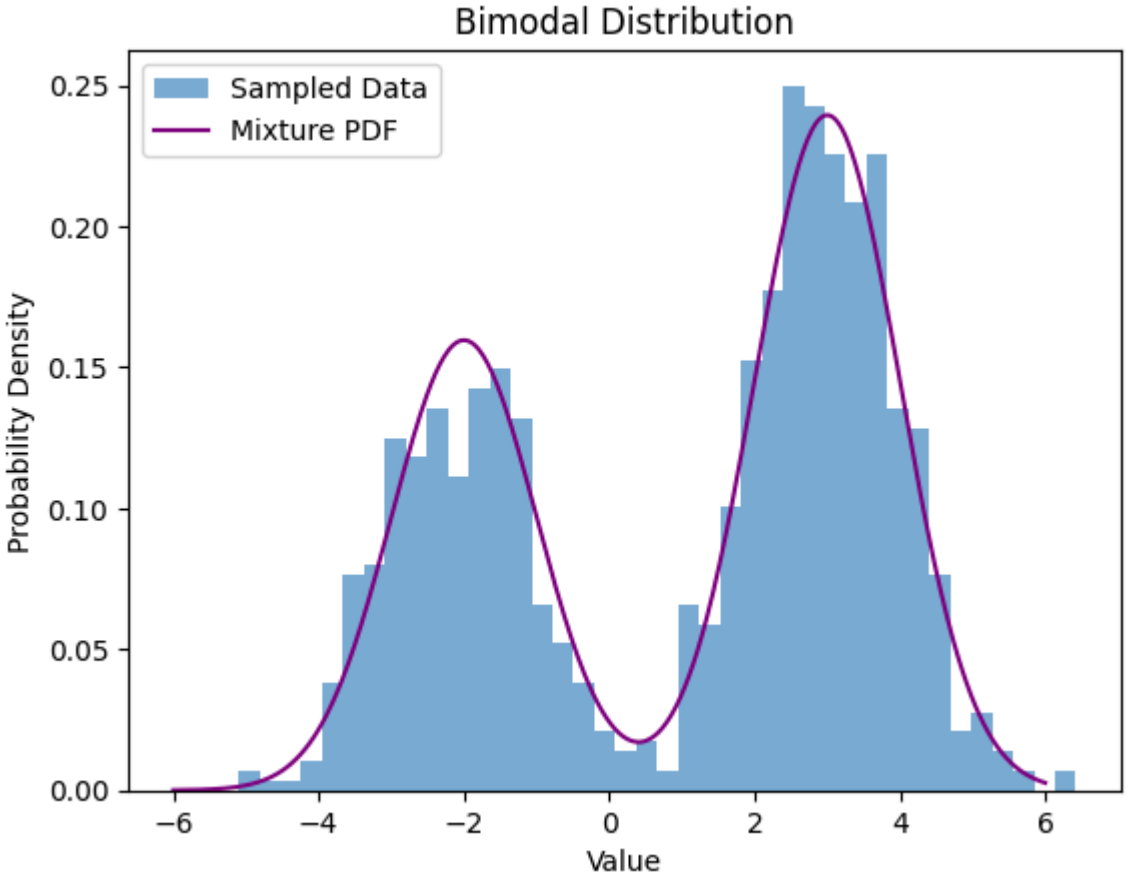
where w1 and w2 are mixing weights with $(w_1 + w_2 = 1)$ and μ_1, σ_2, μ_2 and σ_2 define the two Gaussian components. Repeat step (b),(c) for these data sets

PART (A)

In [443…

```
import numpy as np
import matplotlib.pyplot as plt

# parameters
N = 1000
mu1, sigma1 = -2, 1
mu2, sigma2 = 3, 1
w1, w2 = 0.4, 0.6   # mixing weights (must sum to 1)

# choose which Gaussian to sample from
choices = np.random.rand(N)

data = np.where(choices < w1,
                np.random.normal(mu1, sigma1, N),
                np.random.normal(mu2, sigma2, N))


# plot histogram
plt.hist(data, bins=40, density=True, alpha=0.6, label="Sampled Data")

# theoretical mixture PDF
x_vals = np.linspace(-6, 6, 400)

pdf1 = w1*(1/(np.sqrt(2*np.pi)*sigma1))*np.exp(-(x_vals-mu1)**2/(2*sigma1**2))
pdf2 = w2*(1/(np.sqrt(2*np.pi)*sigma2))*np.exp(-(x_vals-mu2)**2/(2*sigma2**2))

plt.plot(x_vals, pdf1+pdf2, 'purple', label="Mixture PDF")

plt.xlabel("Value")
```

```
plt.ylabel("Probability Density")
plt.title("Bimodal Distribution")
plt.legend()
plt.show()
```

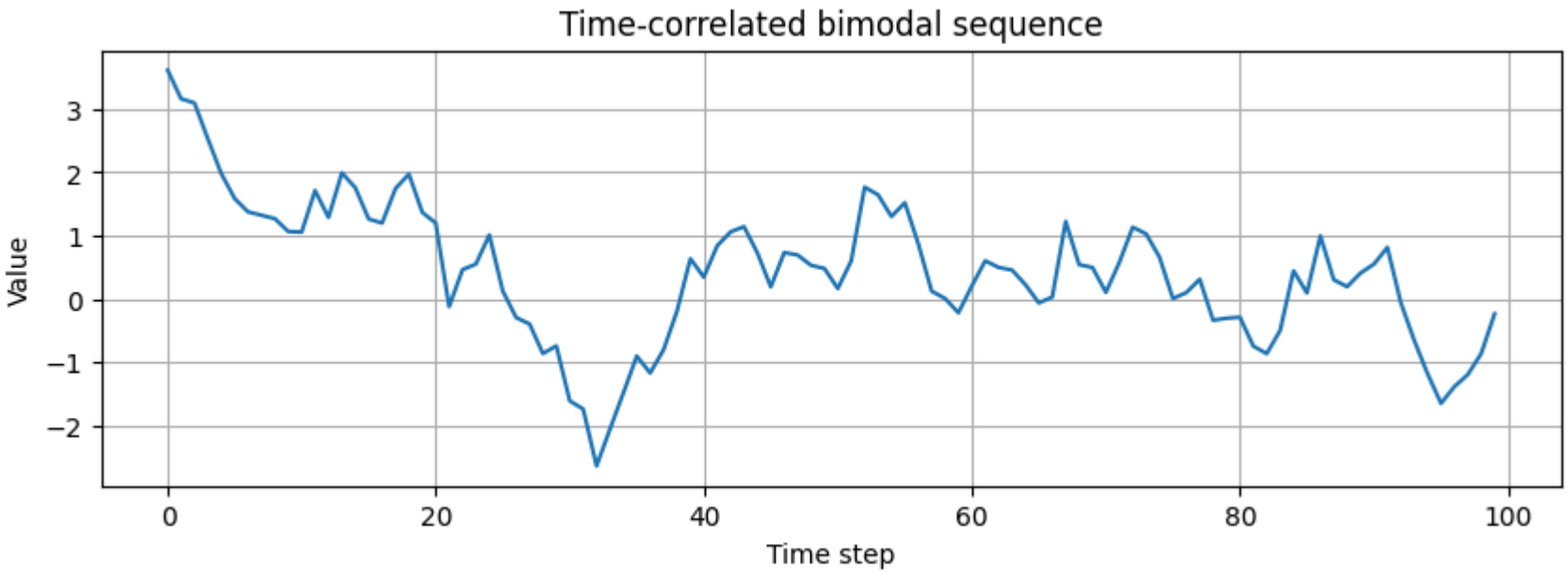


A bimodal distribution was generated by sampling from a mixture of two Gaussian distributions with probabilities w_1 and w_2. This produces two peaks centered at μ_1 and μ_2. The histogram agrees with the theoretical mixture probability density function.

PART (B)

In [470…

```
T = 100        # number of time steps
tau = 10        # correlation time
dt = 1

X = np.zeros((T, N))
X[0] = data

for t in range(1, T):
    noise = np.random.normal(0, np.sqrt(dt), N)
    X[t] = X[t-1] - (X[t-1]/tau)*dt + np.sqrt(2/tau)*noise

plt.figure(figsize=(10, 3))
plt.plot(X[:,0])
plt.xlabel("Time step")
plt.ylabel("Value")
plt.title("Time-correlated bimodal sequence")
plt.grid(True)
plt.show()
```

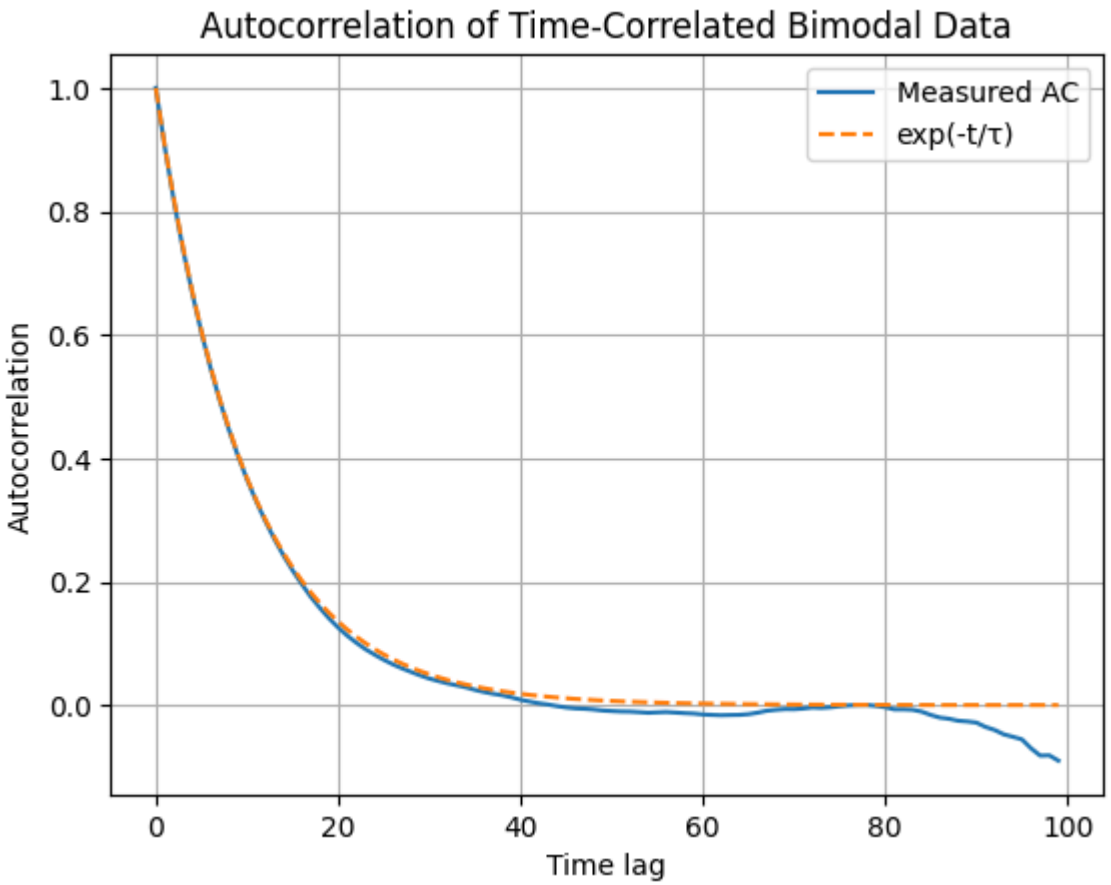


PART (C)

In [474…

```
T, N = X.shape

max_lag = 100
lags = np.arange(max_lag)

#remove mean
mean_t = np.mean(X, axis=1, keepdims=True)
fluct = X - mean_t

denominator = np.mean(fluct**2)

AC = []

for lag in lags:
    if lag == 0:
        AC.append(1)
```

```
    else:
        AC.append(np.mean(fluct[:-lag] * fluct[lag:]) / denominator)

AC = np.array(AC)

# theoretical exponential decay
tau = 10
theory = np.exp(-lags/tau)

# plot
plt.plot(lags, AC, label="Measured AC")
plt.plot(lags, theory, '--', label="exp(-t/τ)")
plt.xlabel("Time lag")
plt.ylabel("Autocorrelation")
plt.title("Autocorrelation of Time-Correlated Bimodal Data")
plt.legend()
plt.grid()
plt.show()
```


Autocorrelation of Time-Correlated Bimodal Data
Measured AC
exp(-t/τ)
Autocorrelation
Time lag
1.0
0.8
0.6
0.4
0.2
0.0
0
20
40
60
80
100